



American International University- Bangladesh (AIUB)

Faculty of Engineering (EEE)

Course Name:	Microprocessor and Embedded Systems	Course Code:	COE3104
Semester:	Summer 2024-2025	Sec:	K
Lab Instructor:	Md. Ali Noor		

Experiment No:	05
Experiment Name:	Familiarization of assembly language program in a microcontroller.

Group Members	Name	ID
	1. Basudeb Kundu	23-50856-1
	2. Prohlad Chandra Das	23-50922-1
	3. Debashis Kumar Das	23-50953-1
	4. Indronill Dutta Nill	23-50974-1
	5. Nafiur Rahman Nirob	23-50991-1

Performance Date:	6-8-25	Due Date:	08-08-25
--------------------------	--------	------------------	----------

Marking Rubrics (to be filled by Lab Instructor)

Category	Proficient [6]	Good [4]	Acceptable [2]	Unacceptable [1]	Secured Marks
Theoretical Background, Methods & procedures sections	All information, measures and variables are provided and explained.	All Information provided that is sufficient, but more explanation is needed.	Most information correct, but some information may be missing or inaccurate.	Much information missing and/or inaccurate.	
Results	All of the criteria are met; results are described clearly and accurately;	Most criteria are met, but there may be some lack of clarity and/or incorrect information.	Experimental results don't match exactly with the theoretical values and/or analysis is unclear.	Experimental results are missing or incorrect;	
Discussion	Demonstrates thorough and sophisticated understanding. Conclusions drawn are appropriate for analyses;	Hypotheses are clearly stated, but some concluding statements not supported by data or data not well integrated.	Some hypotheses missing or misstated; conclusions not supported by data.	Conclusions don't match hypotheses, not supported by data; no integration of data from different sources.	
General formatting	Title page, placement of figures and figure captions, and other formatting issues all correct.	Minor errors in formatting.	Major errors and/or missing information.	Not proper style in text.	
Writing & organization	Writing is strong and easy to understand; ideas are fully elaborated and connected; effective transitions between sentences; no typographic, spelling, or grammatical errors.	Writing is clear and easy to understand; ideas are connected; effective transitions between sentences; minor typographic, spelling, or grammatical errors.	Most of the required criteria are met, but some lack of clarity, typographic, spelling, or grammatical errors are present.	Very unclear, many errors.	
Comments:				Total Marks (Out of)::	

Title: Familiarization of assembly language program in a microcontroller.

Objective:

In this experiment, the main objective is to learn how to write an assembly program for a blink LED program in a microcontroller.

Apparatus:

- 1) Arduino Uno
- 2) Arduino IDE
- 3) One Led
- 4) One 220 ohm resistor
- 5) PC having Intel Microprocessor

Experimental Setup:

PART 1: Blink an LED

Picture for LED blink system using an Arduino Microcontroller

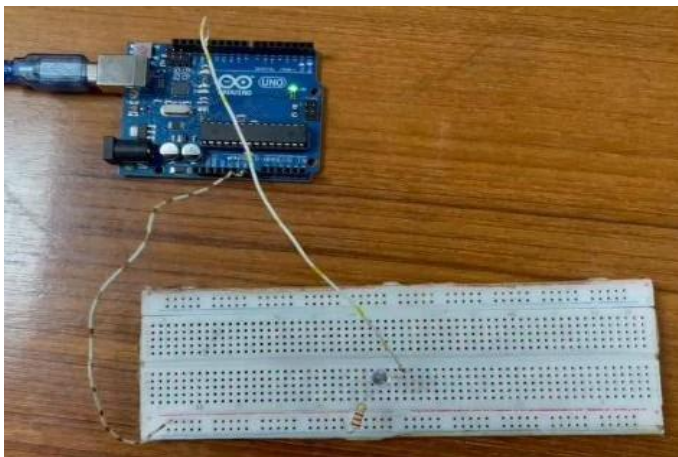


Figure 1

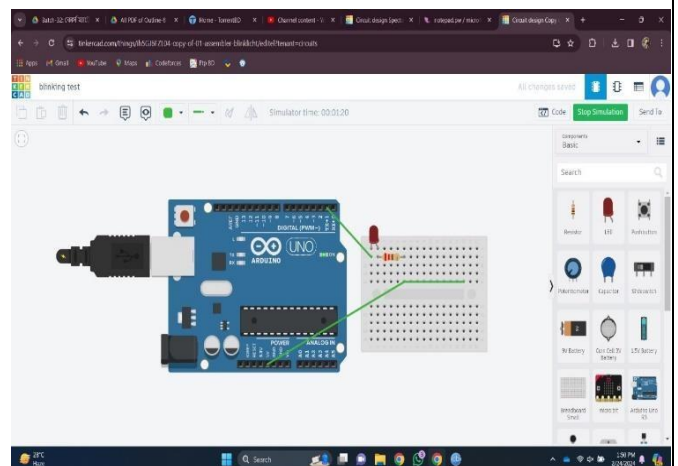


Figure 2

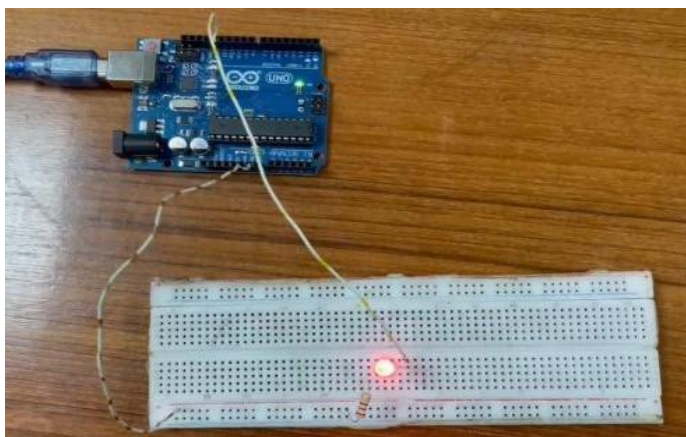


Figure 3

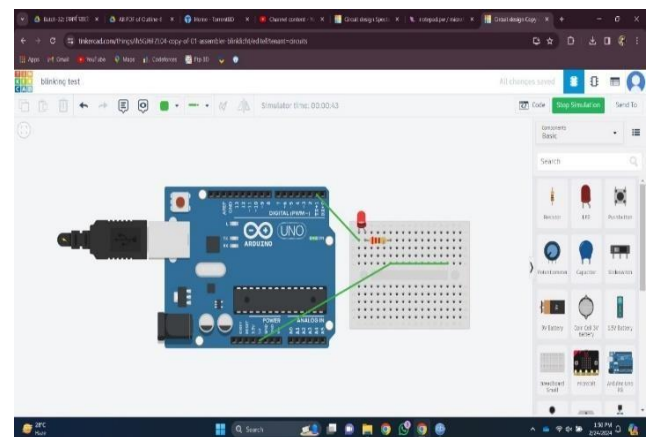


Figure 4

Code:

Code for LED blink system using an Arduino Microcontroller

PART 1: Blink an LED

The .ino file:

```
extern "C"
{
  void start();
  void led(byte);
}
//-----

void setup()
{
  start();
}
//-----

void loop()
{
  led(1);
  led(0);
}
```

The .S file:

```
#define__ SFR_OFFSET 0x00
#include "avr/io.h"
.global start
.global led
start:
  SBI DDRB, 5 ;set PB5 (D13) as o/p
  RET ;return to setup() function
led:
  CPI R24, 0x00 ;value in R24 passed by caller compared with 0
  BREQ ledOFF ;jump (branch) if equal to subroutine ledOFF
  SBI PORTB, 5 ;set D13 to high
  RCALL myDelay
  RET ;return to loop() function
ledOFF:
```

```

CBI PORTB, 5 ;set D13 to low
RCALL myDelay
RET ;return to loop() function
.equ delayVal, 10000;initial count value for inner loop
myDelay:
    LDI R20, 100 ;initial count value for outer loop
outerLoop:
    LDI R30, lo8(delayVal) ;low byte of delayVal in R30
    LDI R31, hi8(delayVal) ;high byte of delayVal in R31
innerLoop:
    SBIW R30, 1 ;subtract 1 from 16-bit value in R31, R30
    BRNE innerLoop ;jump if countVal not equal to 0
    SUBI R20, 1 ;subtract 1 from R20
    BRNE outerLoop ;jump if R20 not equal to 0
    RET

```

Code for Traffic Light system using an Arduino Microcontroller

Part 2: Traffic Light

The .ino file:

```

//-----
// C Code: RGB LED ON/OFF via Buttons
//-----
extern "C"
{
    void start();
    void btnLED();
}
//-----
void setup()
{
    start();
}
//-----
void loop()

```

```

{
    btnLED();
}

The .S file:

;-----
; Assembly Code: RGB LED ON/OFF via Buttons
;-----

#define __SFR_OFFSET 0x00
#include "avr/io.h"
;-----

.global start
.global btnLED

;=====
==
start:
    SBI  DDRB, 4      ;set PB4 (pin D12 as o/p - red LED)
    SBI  DDRB, 3      ;set PB3 (pin D11 as o/p - green LED)
    SBI  DDRB, 2      ;set PB2 (pin D10 as o/p - blue LED)
    CBI  DDRD, 2      ;clear PD2 (pin D02 as i/p - red button)
    CBI  DDRD, 3      ;clear PD3 (pin D03 as i/p - green button)
    CBI  DDRD, 4      ;clear PD4 (pin D04 as i/p - blue button)
    RET              ;return to setup()

;-----

btnLED:
    SBIC PIND, 2      ;skip next statement if red button not pressed
    RJMP redledON     ;jump to label redledON
    SBIC PIND, 3      ;skip next statement if green button not pressed
    RJMP greenledON   ;jump to label greenledON
    SBIC PIND, 4      ;skip next statement if blue button not pressed
    RJMP blueledON    ;jump to label blueledON
    RJMP btnLED       ;return to label btnLED

;-----

redledON:
    LDI  R21, 10      ;initial value of counter R21
redagain:

```

```

SBI  PORTB, 4      ;turn ON red LED
RCALL myDelay      ;call subroutine myDelay
CBI  PORTB, 4      ;turn OFF red LED
RCALL myDelay      ;call subroutine myDelay
SUBI R21, 1        ;decrement counter by 1
BRNE redagain      ;loop if counter not zero
RJMP btnLED        ;return to label btnLED

;-----
greenledON:
    LDI R21, 10     ;initial counter R21 count value
greenagain:
    SBI  PORTB, 3    ;turn ON green LED
    RCALL myDelay    ;call subroutine myDelay
    CBI  PORTB, 3    ;turn OFF green LED
    RCALL myDelay    ;call subroutine myDelay
    SUBI R21, 1      ;decrement counter by 1
    BRNE greenagain  ;loop if counter not zero
    RJMP btnLED      ;return to label btnLED

;-----
blueledON:
    LDI R21, 10     ;initial counter R21 count value
blueagain:
    SBI  PORTB, 2    ;turn ON blue LED
    RCALL myDelay    ;call subroutine myDelay
    CBI  PORTB, 2    ;turn OFF blue LED
    RCALL myDelay    ;call subroutine myDelay
    SUBI R21, 1      ;decrement counter by 1
    BRNE blueagain  ;loop if counter not zero
    RJMP btnLED      ;return to label btnLED

;-----
.equ  delayVal, 10000 ;equate delayVal with initial count value
;----- ;for inner loop
myDelay:
    LDI R20, 90      ;initial count value for outer loop
outerLoop:

```

```
LDI R30, lo8(delayVal) ;low byte of delayVal in R30
LDI R31, hi8(delayVal) ;high byte of delayVal in R31
innerLoop:
    SBIW R30, 1          ;subtract 1 from 16-bit value in R31, R30
    BRNE innerLoop      ;jump if countVal not equal to 0
    ;-----
    SUBI R20, 1         ;subtract 1 from R20
    BRNE outerLoop      ;jump if R20 not equal to 0
```

Discussion and Conclusion:

In this lab experiment the assembly language program executed on the microcontroller allowed us a deeper understanding of low-level programming concepts. Through the direct manipulation of registers and memory addresses, we gained insight into the fundamental operations of the microcontroller. Writing assembly code enabled us to control the hardware at a granular level, facilitating efficient utilization of resources and precise execution of instructions. By dissecting the assembly code, we could comprehend the intricacies of instruction execution, including fetching, decoding, and executing operations. This hands-on experience enhanced our comprehension of processor architecture and instruction set architecture (ISA), which are essential for optimizing code performance and troubleshooting potential issues. Moreover, the assembly language program provided valuable insights into the real-time behavior of the microcontroller. We observed how different instructions affected program flow and data manipulation, contributing to a holistic understanding of embedded systems programming.

In conclusion, the familiarity gained with assembly language programming in a microcontroller environment was invaluable. This experience not only reinforced theoretical concepts but also instilled practical skills essential for embedded systems development. By bridging the gap between hardware and software, we acquired a deeper appreciation for the underlying mechanisms driving microcontroller functionality. Moving forward, the knowledge and skills acquired from this lab experiment will serve as a solid foundation for tackling more complex embedded systems projects and optimizing code efficiency in real-world applications.

Reference(s):

1. Circuit Digest Website, [Online: March 21, 2022], [Cited: June 10, 2023] Available: <https://circuitdigest.com/article/everything-you-need-to-know-about-arduino-uno-board-hardware>
2. MicroPi, Program the Arduino Uno in Assembly Language [Online: March 23, 2021], [Cited: June 24, 2023] Available: <https://micropi.wordpress.com/2021/03/23/program-the-arduino-uno-in-assembly-language>

3. Arduino CC Website, [Online] [Cited: June 10, 2023] Available:
<https://docs.arduino.cc/hardware/uno-rev3>
4. AIUB Microprocessor and Embedded Systems Lab Manual 5